

LocalAI Agent: design and architecture overview

Version 0.15.0 · October 2026

LocalAI Agent is a personal AI agent with the abilities of Meta Muse and Instinct. It runs on the user's own computer: local language models, local memory, local tools. Nothing leaves the machine unless the user approves that specific transfer. This document covers what it does, how it is built, and why each major design choice was made. It ends with a feature comparison against Muse and Instinct.

Companion documents:

- `DESIGN.md`: the deep technical reference.
- `TRUST_UX.md`: trust, observability and controls.
- `SECURITY_REVIEW.md`: security and privacy findings.
- `PRODUCT_ROADMAP.md`: the path to a competitive product.

1. Goals and principles

Principle	What it means	Why
Local-first	Models run in Ollama on the machine. Data sits in one SQLite file and one Markdown file. The UI server listens on 127.0.0.1 only.	Muse and Instinct keep the user's whole life on a vendor VM. The main reason people hold back from these agents is trust. Data that never leaves the laptop removes the biggest objection, and the marginal token cost is zero.
Decide first, generate second	A fast "System 1" decision layer answers typed questions about every input before a large model runs: intent, memorable?, complexity, should we nudge?, how urgent?	Small local models are slow and weaker than frontier models. A ~40 ms decision avoids running the 4B model when the 1.7B one or a direct tool suffices. It also produces numbers the UI can show.
Policy over model	The model only <i>proposes</i> actions. A deterministic policy engine decides from each tool's risk tier whether the action runs, needs approval, or needs fresh approval every time.	A model can be confused or hijacked, but a lookup table can't. The model can never approve its own action.
Everything visible and reversible	Every decision shows its probabilities. Every action, approval and refusal goes into a hash-chained audit log. Memory is a plain file the user can read, edit and forget. Deletes go to the Trash.	Observability is what turns "an AI did something" into "I can see exactly what happened and undo it".
Optional, explicit cloud	Cloud is off by default. When used, it takes the user's own API key and an approval card listing exactly what will be sent.	Some questions need frontier reasoning. Making cloud use an explicit, per-request choice keeps the local promise honest.
Fail closed	Without a sandbox, scripts don't run. Phone access is read-only. After a suspected prompt injection, standing permissions are ignored.	Safety must not depend on the happy path.

2. Architecture

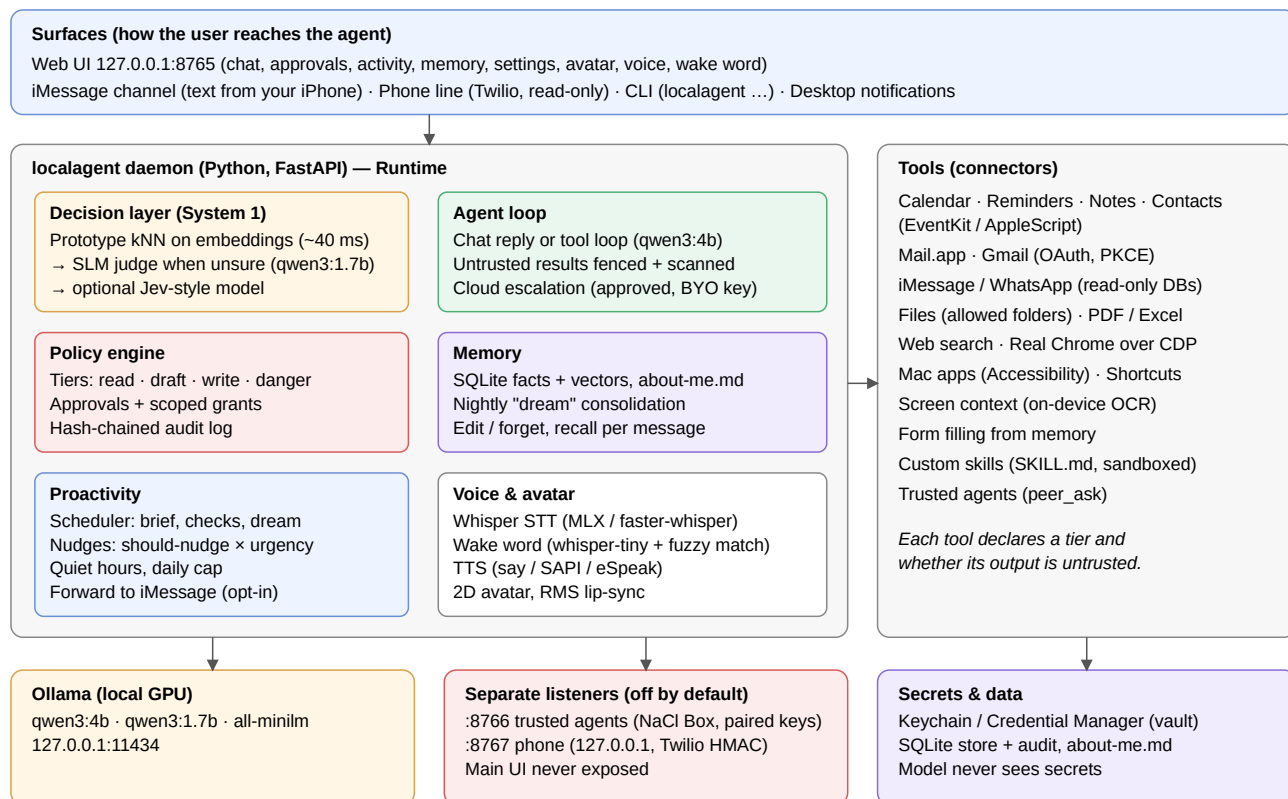


Figure 1. Components. One daemon process; external listeners are separate and off by default.

2.1 A message, end to end

1. **Input** arrives from the UI, the iMessage channel, the phone line, the wake word, or a scheduled job.
2. **Decide**. The prototype classifier embeds the text (all-minilm, 384-dim) and scores it against labelled prototypes (about 40 ms). This gives probabilities for intent (quick answer, task, schedule, computer action, memory...), "is this memorable?" and complexity 1–5. If the top probability is below the confidence threshold, the SLM judge (qwen3:1.7b with JSON-schema output) re-decides. Corrections the user makes in the UI become new prototypes immediately.
3. **Recall** the relevant memories (vector search plus `about-me.md`).
4. **Route**. Small talk goes to the fast model. Tasks go to the tool loop on the main model, with only the tools relevant to the intent. Complexity 5 or "think harder" may propose cloud escalation, behind an approval.
5. **Act**. The tool loop uses Ollama native tool calling. For each proposed call, the policy engine applies the tool's tier for these arguments:
 - **read/draft** tiers run;
 - **write** needs approval, unless the user granted a standing scope;
 - **danger** needs a fresh one-time approval.

Results from other people (mail, chats, web pages, screen, peers) are fenced as data and scanned for injection. A hit *taints* the run, and standing grants stop applying. 6. **Answer** is streamed over SSE with tool chips, approval cards, and the decision chips with their probabilities. 7. **Remember**. If the input is memorable, facts are extracted and deduplicated. Every step is written to the audit log.

2.2 Code map

Area	Module	Notes
HTTP + UI	<code>server.py</code> , <code>ui/</code>	FastAPI, vanilla JS, no build step, SSE
Wiring	<code>runtime.py</code> , <code>config.py</code> , <code>cli.py</code>	One <code>Runtime</code> object owns everything
Decisions	<code>decide/</code>	prototype, SLM judge, System-One client, router, typed questions
Agent	<code>agent/chat.py</code> , <code>agent/actions.py</code> , <code>agent/cloud.py</code>	turn handling, tool loop, cloud
Safety	<code>policy/engine.py</code> , <code>safety/injection.py</code> , <code>vault.py</code>	tiers, grants, audit, injection guard, secrets
Memory	<code>memory/store.py</code>	SQLite + numpy vectors, identity file
Proactive	<code>proactive/</code>	scheduler, jobs, nudges
Connectors	<code>connectors/</code>	one file per integration
Channels	<code>channels/imessage.py</code> , <code>channels/phone.py</code> , <code>peers_server</code> (in <code>connectors/peers.py</code>)	external surfaces
Voice	<code>voice/</code>	STT, TTS, wake word

3. Key design decisions and why

3.1 Models and runtime

Decision	Alternatives considered	Why this one
Ollama as model runtime	llama.cpp directly, MLX-LM, LM Studio	Ollama has one install everywhere, native tool calling and model management, and uses Metal/CUDA without extra work. The cost is a second process.
qwen3:4b main + 1.7b fast	8B models, Llama 3.x, Gemma	This fits 16 GB alongside the OS and apps with headroom for the KV cache. Tool calling is reliable for single-step actions. Two sizes let the decision layer pay for the big model only when needed.
Prototype kNN decision layer with an SLM fallback	Fine-tuned classifier, the LLM decides everything	It needs no training step. Corrections apply instantly. It is deterministic and takes about 40 ms. The probabilities are explainable ("closest examples were..."). The SLM judge covers the long tail.
Jev-style typed questions (choice / yes-no / score)	Free-form JSON from the LLM	Typed questions give numbers the policy and the UI can use: thresholds for nudges, routing by complexity. They are also how dedicated System-1 models work, so one can be swapped in later.
Local Whisper (MLX on Apple Silicon, faster-whisper elsewhere)	Cloud speech APIs, Apple dictation	Audio never leaves the machine. MLX uses the GPU/ANE. faster-whisper is the fastest CPU option.
Wake word via whisper-tiny on short speech bursts	Porcupine, openWakeWord	There's no extra model licence or training per name. Any agent name works with fuzzy phonetic matching. The browser VAD keeps it cheap.

3.2 Actions and safety

Decision	Alternatives considered	Why this one
Four risk tiers declared by each tool , sometimes per call (<code>risk(args)</code>)	Ask the model how risky something is	The model can be manipulated, but a static table can't. Per-call risk lets a click on "Pay" be <i>danger</i> while a click on "Next" is <i>draft</i> .
Approvals with six scopes (once, task, session, 1 h, 24 h, always); danger is once-only	Always ask; never ask	This matches Muse's model. Too many prompts train people to click "yes" blindly. Scopes let routine actions flow while irreversible ones always stop.
Hash-chained audit log	Plain log table	Edits to past rows are detectable (see the security review for the limits). It gives the user one place to see everything the agent did.
Injection guard: fence + pattern scan + taint	Rely on the model; a second "quarantine" LLM	It is deterministic and costs nothing at runtime. Tainting removes standing grants, so a hijacked run must get past a human approval. The security review notes where this is not yet enough.
Secrets in the OS keychain , never in config or prompts	<code>.env</code> file, config JSON	OS-level protection. Tools see "connected / not connected" only.
Real Chrome over CDP with the agent's own profile, with an overlay showing each click	Playwright's bundled Chromium	Fewer bot blocks and a familiar browser, and the user can watch every action, as in Muse. Bundled Chromium failed on users' Macs.
Skills run in an OS sandbox (sandbox-exec, bubblewrap), refused if none	Run scripts directly	User-written or downloaded scripts are the highest-risk code path.

3.3 Integrations

Decision	Alternatives considered	Why this one
AppleScript / EventKit for Apple apps	Cloud CalDAV/IMAP	The data is already on the Mac, with no credentials to manage. EventKit expands repeating events correctly.
Read iMessage/WhatsApp from local databases, read-only	Unofficial APIs, bridges	Nothing new leaves the device, and opening the files read-only means the databases can't be corrupted. Sending iMessage goes through Messages.app; WhatsApp replies are prepared for the user to send.
iMessage channel in "note to self" mode by default	Dedicated Apple ID only	It works with one Apple ID: you text yourself "Ari, ...". A second Apple ID is supported for a cleaner thread.
Gmail via the user's own OAuth client (PKCE, loopback)	A shared vendor client	No vendor server sits in the middle, and tokens go into the user's keychain. The cost is a 10-minute setup.
Trusted agents: paired keys, NaCl Box, separate listener	A central relay server	It is end-to-end encrypted with no operator. Invite codes carry the public key, so there is no PKI. Replay protection uses nonce + timestamp.
Phone line via Twilio, read/draft tools only	Local SIP, full tool access	A phone call can't show an approval card, so nothing irreversible may happen by phone. The listener is 127.0.0.1-only and is reached through a user-run tunnel. Every request is HMAC-verified.
Cloud escalation via the official Anthropic SDK, behind an approval	Silent fallback to the cloud	The user sees exactly what will be sent, every time, and uses their own key.

3.4 Platform and packaging

- A **single Python package** installed with pipx. The installers (`install.sh` for macOS/Linux, `install.ps1` for Windows) add Ollama, the models and the optional add-ons. Upgrades keep all data.
- **One code base for macOS, Windows and Linux.** Platform differences sit behind small selectors: STT/TTS, notifications, sandbox, autostart, Chrome discovery. Apple-only connectors are not emulated, and `doctor` says what needs a Mac.
- **Separate listeners for anything reachable from outside** (peers 8766, phone 8767). A bug in an external surface then can't expose the main UI API.

4. Feature list

✓ shipped · 🟡 partial · ✗ not available

Area	Feature	Status	Notes
Core	Streaming chat, persona (name, tone), model manager	✓	
	Decision layer with visible probabilities and corrections	✓	intent, memorable, complexity, should-nudge, urgency
	Optional cloud model ("think harder")	✓	BYO Anthropic key, approval per request
Memory	Persistent memory, <code>about-me.md</code> identity, edit/forget	✓	
	Nightly "dream" consolidation	✓	runs when the machine is awake
Apple apps	Calendar, Reminders, Notes, Mail, Contacts	✓	macOS
	Mac apps (Accessibility) and Shortcuts	✓	pressing buttons asks first
	Screen context (on-device OCR, private apps skipped, 2 h retention)	✓	opt-in, macOS
Messages	Read iMessage/SMS and WhatsApp; waiting-on-you threads	✓	macOS
	Reply by iMessage (approved); WhatsApp reply prepared	✓	
Mail	Gmail: search, read, follow-ups, drafts, send	✓	OAuth, keychain
Web	Web look-ups (search + read source page)	✓	prices, hours, shops
	Browser automation in real Chrome with visible clicks	✓	submit/pay always asks
	Form filling from memory	✓	never submits for you
	Checkout with payment	🟡	user completes payment; no virtual cards
Files	Search, list, move, trash, open; create PDF/Excel	✓	allowed folders only
Proactive	Morning brief, nudges (events, reminders, unreplied mail/chats)	✓	quiet hours, daily cap
Channels	iMessage channel (text the agent, approve by "yes")	✓	macOS
	Phone line (call your agent)	✓	Twilio, read/draft only
	WhatsApp channel	✗	no official personal API
	Mobile app	✗	phone reaches the agent via iMessage/phone

Area	Feature	Status	Notes
Voice	Push-to-talk, conversation mode, wake word "Hey Ari"	✓	on-device
	Avatar with lip-sync, read-aloud	✓	2D
Agents	Trusted-agent network (free/busy, messages, questions)	✓	E2E encrypted, paired
	Concurrent sub-agents	●	tasks queue on one local model
Extensibility	Custom skills (SKILL.md + sandboxed scripts)	✓	Windows: instruction-only
Safety	Tiers, scoped approvals, audit log, injection guard, vault	✓	
Platforms	macOS (Apple Silicon), Windows, Linux	✓ / ●	Windows/Linux without Apple apps

5. Comparison with Meta Muse and Instinct

Sources: public launch coverage and reviews (September–October 2026), summarised in `PLAN.md §1`. ✓ yes · ● partial · ✗ no.

5.1 Capabilities

Capability	Meta Muse	Instinct	LocalAI Agent
Chat assistant	✓ app, web, WhatsApp	✓ iMessage/WhatsApp/phone, no app	✓ local web app, iMessage, phone
Underlying model	Frontier (Muse Spark), cloud	Frontier, cloud	Local 1.7B–4B; optional cloud on approval
Persistent memory + identity view + forget	✓	✓	✓ (plain file, editable)
Nightly memory consolidation	✓ "dream"	●	✓
Email	✓	✓	✓ Mail.app + Gmail
Calendar / reminders / notes	✓	✓	✓ (Apple apps)
Read your messages (iMessage, WhatsApp)	● Mac app	✓	✓
Acts in native desktop apps	✓ Mac app	✗	✓ (Accessibility + Shortcuts)
Browser agent you can watch	✓	●	✓ real Chrome with click overlay
Shopping with checkout	✓ partners + one-time cards	✓ one-time cards	● stops before payment
Partner connectors (1,500+)	✓	●	✗ (skills + browser instead)
Fill forms from memory	✓	✓	✓ (review before submit)
Documents (PDF/spreadsheet)	✓	●	✓
Proactive nudges / follow-ups	●	✓ texts and calls first	✓ notifications + iMessage
Screen context	✗	✓	✓ on-device OCR, opt-in

Capability	Meta Muse	Instinct	LocalAI Agent
Ambient audio / location context	✗	✓	✗ (by choice)
Text the agent from your phone	✓ WhatsApp	✓ iMessage	✓ iMessage
Call the agent	✗	✓	✓ (Twilio, read-only)
Agent calls businesses for you	✗	✓	✗ (excluded)
Agent-to-agent with people you trust	✗	✓	✓ E2E encrypted, no server
Voice conversation, wake word	✓	✓ calls	✓ on-device
Avatar	● teased video avatar	✗	✓ 2D lip-sync
Custom skills	✓	✗	✓ sandboxed
Concurrent sub-agents	✓ cloud VM	✓	● one model at a time
Works with the computer off	✓	✓	✗ (needs the machine awake)
Platforms	Web, iOS, Android, Mac	Messaging apps	macOS, Windows, Linux

5.2 Trust, privacy and cost

Property	Meta Muse	Instinct	LocalAI Agent
Where your data lives	Vendor cloud VM (US)	Vendor cloud	Your computer only
Who can read it	Vendor (policy-limited)	Vendor	You (plus OS-level access)
Model sees your passwords/cards	No (vault)	No (vault)	No (keychain; never in prompts)
Approvals with scopes	✓ 5 scopes	●	✓ 6 scopes; danger always once
Audit trail	✓	●	✓ hash-chained, local
Visible reasoning for decisions	✗	✗	✓ probabilities per decision
Prompt-injection defences	Undisclosed	Had a public incident	Fence + scan + taint (see review)
Can enter binding agreements	No	Yes (per ToS)	Never
Data after you quit	Vendor retention policy	Reported retained	Delete the folder; it's gone
Ongoing cost	Subscription / usage	Subscription	\$0 model cost (optional BYO cloud key)
Works offline	✗	✗	✓ (except web/Gmail features)

5.3 Honest gaps

- **Reasoning quality:** 4B local models trail frontier models on long multi-step plans. This is mitigated by the decision router, scripted skills, short tool chains, and optional cloud escalation.
- **Availability:** the agent stops when the laptop sleeps. Muse and Instinct run on servers that never sleep.
- **Commerce:** there are no partner integrations or virtual cards, so the user completes payments.
- **Mobile:** there is no native phone app. The phone reaches the agent through iMessage or a call.

PRODUCT_ROADMAP.md addresses each of these.